Lany : bilan backend (étapes 4.1 et 4.2)

Résumé de ce qui est terminé côté Spring Boot, avec rappels sur Flyway, `mvn test` et JUnit.

1. Ce qui est fait

Élément	État
Projet Maven (Spring Boot 4.1.x, Java 21, `com.lany`, `lany-backend`)	Créé
Dépendances : Web, Data JPA, Validation, Flyway (+ module PostgreSQL), PostgreSQL Driver, Testcontainers, Actuator	En place (Spring Security reporté à l'étape d'authentification)
Base `lany_dev` et utilisateur dédié `lany`	Créés
Migrations `V1__init.sql` et `V2__add_version_columns.sql`	Appliquées par Flyway au démarrage
`application.yml` (commun) et `application-dev.yml` (local, non committé)	Configurés
Démarrage et `GET /actuator/health` → `UP`	Validés
`mvn test` avec Docker	Validé

Aucune fonctionnalité métier à ce stade : l'ossature est validée.

2. Flyway : comment ça fonctionne

Les deux dépendances

Dépendance	Rôle
Starter Flyway	Intègre Flyway à Spring Boot : il s'active tout seul au démarrage et réutilise la `DataSource` configurée. Embarque le moteur `flyway-core`.

Dépendance	Rôle
Module Flyway pour PostgreSQL	Support spécifique de PostgreSQL. Sans lui, Flyway signale que la base n'est pas supportée.

Ce qui se passe à chaque démarrage

1. Flyway se connecte à la base avec les identifiants de `application-dev.yml`.
2. Il crée la table `flyway_schema_history` si elle n'existe pas.
3. Il scanne `src/main/resources/db/migration` et liste les scripts `V<version>__<description>.sql`.
4. Il compare avec l'historique : pour chaque script déjà appliqué, il vérifie que le **checksum** (empreinte du contenu) n'a pas changé.
5. Il applique, dans l'ordre des versions, les scripts **pas encore appliqués**, et note chacun dans l'historique.
6. **Ensuite seulement**, Hibernate démarre et valide les entités (`ddl-auto: validate`). Spring Boot impose cet ordre : Flyway d'abord, JPA ensuite.

La table `flyway_schema_history`

Chaque ligne = une migration appliquée : `version`, `description`, `script`, `checksum`, `installed_on`, `success`. C'est la mémoire de Flyway : sans elle il ne saurait pas ce qui a déjà été exécuté.

Règles à retenir

Règle	Pourquoi
Ne jamais modifier un script déjà appliqué	Le checksum change et Flyway refuse de démarrer. Toute évolution passe par un nouveau fichier (`V3__...`).
Nom exact : `V` + version + `__` (deux underscores) + description	Un nom mal formé est ignoré ou rejeté.
Les versions se comparent comme des nombres	`V10` passe après `V2`.

Règle	Pourquoi
Pas d'exécution dans le désordre	Un script de version inférieure ajouté après coup est refusé par défaut.
Une base non vide sans historique fait échouer Flyway	Il ne sait pas ce qu'elle contient. D'où la base neuve `lany_dev`.
En cas d'échec d'une migration	PostgreSQL gère les changements de schéma dans une transaction : l'échec annule en général tout le script. On corrige puis on relance.

Pourquoi c'est utile pour Lany

- Le schéma est **versionné dans Git** avec le code.
- Un nouvel environnement (autre machine, test, production) obtient **exactement le même schéma** automatiquement.
- Les **tests exécutent les mêmes migrations** sur un PostgreSQL vierge : une migration cassée est détectée avant de tourner en production.

3. `mvn test` : comment ça marche

Ce que fait la commande

Maven suit un **cycle de vie** composé de phases : `validate` → `compile` → `test-compile` → `test` → `package` … Lancer `mvn test` exécute toutes les phases **jusqu'à `test` incluse** (pas de jar produit). Le plugin **Surefire** exécute alors les classes de test, par défaut celles dont le nom finit par `Test` ou `Tests` (ou commence par `Test`), situées dans `src/test/java`. Les rapports sont écrits dans `target/surefire-reports`.

Est-ce généré par défaut ?

Oui. Spring Initializr crée une classe de test minimale (`LanyBackendApplicationTests`) contenant un test vide `contextLoads()`, annoté `@SpringBootTest`. Comme tu as choisi Testcontainers, il génère aussi en général une classe de configuration pour le conteneur PostgreSQL (`TestcontainersConfiguration`) et un lanceur de test (`TestLanyBackendApplication`). Les noms exacts dépendent de la version : regarde dans `src/test/java`.

Ce que vérifie `contextLoads()`

Le test passe **si l'application démarre**, donc :

1. Testcontainers lance un **PostgreSQL jetable** via Docker.
2. La configuration `@ServiceConnection` branche automatiquement l'application sur ce conteneur (aucune URL à écrire).
3. Flyway applique **V1 et V2** sur cette base vierge.
4. Hibernate valide les entités contre le schéma.
5. Le conteneur est détruit à la fin.

Ce test **n'utilise ni `lany_dev` ni le profil `dev`** : il est indépendant de ta base locale. C'est pourquoi Docker doit tourner.

Lent ou rapide ?

Un test `@SpringBootTest` démarre toute l'application avec un conteneur : quelques secondes. On en écrit donc **peu**. Les règles de calcul, elles, seront testées **sans Spring** pour tourner en quelques millisecondes (voir §4).

4. JUnit : comment ça marche

JUnit est le framework de tests unitaires de Java. Spring Boot 4 utilise **JUnit 6** (Jupiter), fourni par le starter de test, avec **AssertJ** pour écrire des vérifications lisibles.

Rôle des parties

Partie	Rôle
JUnit Platform	Découvre et lance les tests (appelée par Surefire).
JUnit Jupiter	L'API que tu utilises : annotations et moteur d'exécution.
AssertJ	Vérifications fluides : `assertThat(x).isEqualTo(y)`.

Structure d'un test : Arrange, Act, Assert

```
@Test
@DisplayName("2 000 Ar par jour pendant 20 jours font 40 000 Ar")
void multiplies() {
    Money daily = Money.of("2000");                    // Arrange : préparer
    Money total = daily.times(20);                     // Act : exécuter
    assertThat(total).isEqualTo(Money.of("40000"));    // Assert : vérifier
}
```

(`Money` sera écrit à l'étape 4.3.)

Annotations utiles

Annotation	Rôle
`@Test`	Marque une méthode de test.
`@DisplayName`	Libellé lisible dans les rapports.
`@BeforeEach`	Code exécuté avant chaque test (préparation commune).
`@Nested`	Regroupe des tests dans une classe interne.
`@ParameterizedTest`	Même test avec plusieurs jeux de valeurs.
`@SpringBootTest`	Démarre le contexte Spring complet (lent, à utiliser avec parcimonie).

Résultat d'un test

Un test **réussit** si aucune exception n'est levée. Un `assertThat(...)` faux lève une erreur d'assertion : le test est **rouge**, avec la valeur attendue et la valeur obtenue.

Deux familles de tests dans Lany

Famille	Exemple	Vitesse	Quantité
Unitaire (sans Spring)	`Money`, calendrier budgétaire, projection	Millisecondes	Beaucoup

Famille	Exemple	Vitesse	Quantité
Intégration (Spring + Testcontainers)	Démarrage, Repositories, contraintes SQL	Secondes	Peu, ciblés

5. Commandes utiles

Commande	Effet
`mvn spring-boot:run -Dspring-boot.run.profiles=dev`	Lance l'application sur `lany_dev`.
`mvn test`	Compile et exécute tous les tests.
`mvn test -Dtest=MoneyTest`	Exécute une seule classe de test.
`mvn clean`	Supprime `target/` (repart d'une compilation propre).
`curl http://localhost:8080/actuator/health`	Vérifie que l'application répond.

6. Points de vigilance

- Ne **jamais committer** `application-dev.yml` (il contient un mot de passe).
- Ne **jamais modifier** `V1` ou `V2` : créer `V3`.
- Docker doit être **lancé** pour `mvn test`.
- Beaucoup de tutoriels en ligne décrivent Spring Boot 3 : avec Boot 4, Jackson 3 et JUnit 6 diffèrent par endroits.